

作业 HW2-栈和队列 实验报告

姓名：毛星博 学号：2351520 日期：2024 年 10 月 17 日

- # 实验报告格式要求按照模板（使用 Markdown 等也请保证报告内包含模板中的要素）
- # 对字体大小、缩进、颜色等不做强制要求（但尽量代码部分和文字内容有一定区分，可参考 vscode 配色）
- # 实验报告要求在文字简洁的同时将内容表示清楚
- # 报告内不要大段贴代码，尽量控制在 20 页以内

1. 涉及数据结构和相关背景

2. 实验内容

2.1 列车进站

2.1.1 问题描述

每一时刻，列车可以从入口进车站或直接从入口进入出口，再或者从车站进入出口。即每一时刻可以有一辆车沿着箭头 a 或 b 或 c 的方向行驶。现在有一些车在入口处等待，给出该序列，然后给你多组出站序列，请你判断是否能够通过上述的方式从出口出来。

2.1.2 基本要求

输入

第 1 行，一个串，进站序列。

后面多行，每行一个串，表示出栈序列

当输入=EOF 时结束

输出

多行，若给定的出栈序列可以得到，输出 yes,否则输出 no。

2.1.3 数据结构设计

图、描述均可

贴出数据结构定义代码（变量、结构体或类）并使用注释辅助说明

```
/// @brief 代表一个栈的数据结构
struct Stack {
    char* base;    ///< 栈底指针
    char* top;     ///< 栈顶指针
    int stack_size; ///< 栈的容量

    /// @brief 构造函数，初始化栈
    /// @param size 栈的大小
    Stack(int size) {
        base = new char[size];
```

```

        top = base;

        stack_size = size;
    }

    /// @brief 向栈中添加元素
    /// @param c 要添加的字符
    void push(char c) {

        if (top - base == stack_size) {

            cout << "stack overflow" << endl;

            return;

        }

        *top++ = c;
    }

    /// @brief 从栈中弹出元素
    /// @return 返回弹出的字符，如果栈为空则返回 0
    char pop() {

        if (top == base) {

            return 0;

        }

        return *--top;
    }

};

```

2.1.4 功能说明（函数、类）

推荐使用代码+注释的形式，清晰描述函数功能、输入内容和输出/返回内容

不要直接贴函数实现代码，请选择关键代码/伪代码展示并用注释辅助说明

```

// 判断栈的状态是否符合输入和输出字符串
void judge_stack(Stack& stack, char* input_str, char* output_str) {

    char* p = input_str;

    char* q = output_str;

    while (*p != 0) { // 遍历输入字符串

        while (*p == *q && *p != 0) { // 输入和输出匹配

            // 跳过相同元素

        }

        if (*p == 0) { // 输入字符串遍历完毕

            break;

        } else {

            char c = stack.pop();

            if (c == 0) { // 栈为空

                stack.push(*p);

                p++;

            } else if (c != *q) { // 不匹配

                //先入栈
            }
        }
    }
}

```

```

        } else {
            q++;
        }
    }
}

while (*q != 0) { // 遍历输出字符串
    char c = stack.pop(); // 弹出栈顶元素
    if (c != *q) {
        // 不匹配
        return;
    }
    q++;
}
// 匹配
return;
}

```

2.1.5 调试分析（遇到的问题解决方法）

在解决列车进站问题的过程中，我遇到的主要难点是如何有效判断给定的出站序列是否是合法的。一开始，我对出站序列的合法性没有清晰的思路，导致在实现过程中产生了一些错误。为了解决这个问题，我进行了多次模拟和手动测试，通过观察不同入站和出站序列的变化，我逐渐找到了规律。

具体来说，我发现可以直接出站的列车应该优先出站，而不能立即出站的列车则需要入栈（即进入车站）。如果当前需要出站的列车与入口处的列车和栈顶的列车都不匹配，那么这个出站序列就是非法的。因此，我在代码中增加了对这些条件的判断，确保在每一步出栈操作之前都进行合法性检查。这种方法不仅让我更清晰地理解了问题的逻辑，也有效地解决了实现中的错误。

2.1.6 总结和体会

通过这个实验，我深刻体会到了栈这种数据结构在解决实际问题中的重要性。列车进站问题虽然相对基础，但它考察了我们对栈的入栈和出栈操作的理解。通过对入栈和出栈过程的模拟，我加深了对栈的特性与操作的理解，特别是在处理先进后出（LIFO）原则时的灵活应用。

此外，这个实验也让我意识到，面对复杂问题时，逐步模拟和手动推演是寻找解决方案的有效方法。通过反复的练习和调试，我的算法思维得到了提升，这将为我今后遇到更复杂的编程问题打下坚实的基础。我相信，这种对基本数据结构的深入理解将有助于我在后续的学习和项目中更轻松地应对各种挑战。

2.2 最长子串

2.2.1 问题描述

已知一个长度为 n ，仅含有字符 '(' 和 ')' 的字符串，请计算出最长的正确的括号子串的长度及起始位置，若存在多个，取第一个的起始位置。

子串是指任意长度的连续的字符序列。

例 1: 对字符串 "(()())()"来说, 最长的子串是"(()())", 所以长度=6, 起始位置是 0。

例 2: 对字符串")())"来说, 最长的子串是")",子串长度=2, 起始位置是 1。

例 3; 对字符串""来说, 最长的子串是"",子串长度=0, 空串的起始位置规定输出 0。

字符串长度: $0 \leq n \leq 1 \times 10^5$

对于 20%的数据: $0 \leq n \leq 20$

对于 40%的数据: $0 \leq n \leq 100$

对于 60%的数据: $0 \leq n \leq 10000$

对于 100%的数据: $0 \leq n \leq 100000$

提示: 查找正确的括号子串可以用栈来实现, 注意会有非法的右括号, 比如例 2 中的第一个右括号。

2.2.2 基本要求

输入

一行字符串。

输出

子串长度, 及起始位置

2.2.3 数据结构设计

图、描述均可

贴出数据结构定义代码(变量、结构体或类)并使用注释辅助说明

```
/// @brief 实现一个动态栈, 支持基本的栈操作
struct Stack {
    int *data;
    int top;
    int size=10;

    /// @brief 构造函数, 初始化栈
    Stack() : top(-1), size(1) {
        data = new int[size]; // 初始化数据数组
    }

    /// @brief 压入元素到栈顶
    /// @param value 要压入栈的值
    void push(int value) {
        if (top == size - 1)
            // 如果栈满, 重新分配内存
            // 压入栈顶
        }

    /// @brief 弹出栈顶元素
    void pop() {
        // 弹出栈顶
    }

    /// @brief 返回栈顶元素
```

```

    /// @return 栈顶的值

    int peek() {

        return data[top]; // 返回栈顶元素

    }

    /// @brief 判断栈是否为空
    /// @return 如果栈为空则返回 true, 否则返回 false

    bool isEmpty() {

        return top == -1; // 判断栈是否为空

    }

};

```

2.2.4 功能说明（函数、类）

推荐使用代码+注释的形式，清晰描述函数功能、输入内容和输出/返回内容

不要直接贴函数实现代码，请选择关键代码/伪代码展示并用注释辅助说明

```

while (cin >> s) { // 读取输入字符串

    if (s==NULL||strlen(s)==0) {

        // 如果没有输入，输出 0 0

        cout << "0 0" << endl;

        return 0;

    }

    int n = strlen(s);

    Stack stack; // 创建栈实例

    stack.push(-1); // 初始化栈，放入-1

    int maxLen = 0;

    int startPos = 0;

    // 遍历输入字符串，计算有效的括号长度

    for (int i = 0; i < n; i++) {

        if (s[i] == '(') {

            // 如果是左括号，压入索引

        }

        else if (s[i] == ')') {

            // 如果是右括号，弹出栈顶

            if (stack.isEmpty()) {

                // 如果栈空，压入当前索引

            }

            else { // 计算当前有效长度

                int currentLen = i - stack.peek();

                if (currentLen > maxLen) {

                    maxLen = currentLen;

                    startPos = stack.peek() + 1;

                }

            }

        }

    }

}
}

```

```

        else {
            ;// 如果是其他字符，跳过
        }
    }

    // 输出最长有效括号的长度和起始位置
    cout << maxLen << " " << startPos << endl;
}

delete[] s; // 释放动态分配的内存

```

2.2.5 调试分析（遇到的问题 and 解决方法）

在解决最长合法括号子串问题的过程中，我遇到的最大挑战是如何准确地定位新的合法括号子串的起始位置。一开始，我尝试通过对左右括号字符进行入栈和出栈的方式来管理括号的匹配关系，但很快我发现这种方法无法有效地记录合法子串的起始位置。由于在出栈时无法直接得知当前的合法起始位置，因此我不得不寻找其他解决方案。

经过反复思考，我决定改用将括号的索引压入栈中的方法。这样做的好处在于，我可以在遇到右括号时，先检查栈中是否存在能匹配的左括号。如果栈为空，说明当前右括号没有匹配的左括号，于是我将其索引压入栈中，标记为新的合法子串的开始位置。接着，如果下一个字符是左括号，这就意味着我找到了一个新的合法子串的起点。而当我找到匹配的左括号时，便可以将其从栈中弹出，并计算当前有效括号的长度。这种方法不仅简化了逻辑，还确保了我能够准确记录并计算合法子串的长度。

2.2.6 总结和体会

通过这道题目，我学到了一个之前未曾考虑过的栈结构使用方式。之前我一直认为栈中的元素必须是要操作的字符，但此次实验让我认识到，栈的元素也可以是索引，这种灵活性为解决问题带来了新的思路。在实际编程中，能够将问题抽象化并找到合适的数据结构进行处理是非常重要的。此外，这道题也让我认识到在解题过程中灵活运用不同的数据结构是解决复杂问题的关键。通过不断尝试和调整我的方法，我不仅找到了问题的解决方案，也提升了自己的逻辑思维能力。这个过程激励我在今后的学习和项目中，继续探索和尝试多种数据结构的应用，以更有效地解决问题。

2.3 布尔表达式

2.3.1 问题描述

计算如下布尔表达式 $(V|V) \& F \& (F|V)$ 其中 V 表示 True, F 表示 False, | 表示 or, & 表示 and, ! 表示 not（运算符优先级 not > and > or）

2.3.2 基本要求

输入：

文件输入，有若干 ($A \leq 20$) 个表达式，其中每一行为一个表达式。表达式有 ($N \leq 100$) 个符号，符号间可以用任意空格分开，或者没有空格，所以表达式的总长度，即字符的个数，是未知的。

对于 20% 的数据，有 $A \leq 5$, $N \leq 20$ ，且表达式中包含 V、F、&、|

对于 40%的数据，有 $A \leq 10$ ， $N \leq 50$ ，且表达式中包含 V、F、&、|、!

对于 100%的数据，有 $A \leq 20$ ， $N \leq 100$ ，且表达式中包含 V、F、&、|、!、(、)

所有测试数据中都可能穿插空格

下载并运行 96.py 生成随机测试数据

输出：

对测试用例中的每个表达式输出“Expression”，后面跟着序列号和“:”，然后是相应的测试表达式的结果（V 或 F），每个表达式结果占一行（注意冒号后面有空格）。

2.3.3 数据结构设计

图、描述均可

贴出数据结构定义代码（变量、结构体或类）并使用注释辅助说明

```
/// @brief 操作符的栈结构体
struct Operater { // 操作符的栈
    char *base; // 栈底指针
    char *top; // 栈顶指针
    int size = 10; // 栈的初始大小

    /// @brief 构造函数，初始化栈
    Operater() {
        base = new char[size]; // 分配初始内存
        top = base; // 初始化栈顶指针
    }

    /// @brief 压栈操作，将字符压入栈中
    /// @param c 要压入栈中的字符
    void push(char c) {
        if (top - base >= size) { // 如果栈满，扩展栈大小
            // 更新栈的大小
        }
        *top = c; // 将字符存入栈顶
        top++; // 更新栈顶指针
    }

    /// @brief 弹栈操作，从栈中弹出字符
    /// @return 弹出的字符，如果栈为空则返回'\0'
    char pop() {...}

    /// @brief 查看栈顶元素，但不弹出
    /// @return 栈顶字符，如果栈为空则返回'\0'
    char peek() {...}
};
```

```
/// @brief 操作数的栈
struct Stack {
    bool *base;
    bool *top;
```

```

int size = 10;

// 构造函数，初始化栈
Stack() {
    base = new bool[size];
    top = base;
}

// 将元素压入栈中
void push(bool c) {...}

// 从栈中弹出元素
bool pop() {...}

// 查看栈顶元素
bool peek() {...}
};

```

2.3.4 功能说明（函数、类）

推荐使用代码+注释的形式，清晰描述函数功能、输入内容和输出/返回内容

不要直接贴函数实现代码，请选择关键代码/伪代码展示并用注释辅助说明

```

// 该函数用于返回给定操作符的优先级
int priority(char c) { // 操作符优先级
    if (c == '|') return 1;
    if (c == '&') return 2;
    if (c == '!') return 3;
    return 0;
}

```

```

// 用于计算布尔表达式的值
// 通过解析输入的字符串，并利用栈来处理操作符和数值
// 支持的操作符包括与、或和取反
while (getline(cin, str)) {
    n++;
    int len = str.length();
    Operator op; // 操作符栈
    Stack num;   // 数值栈

    for (int i = 0; i < len; i++) {
        if (str[i] == 'V' || str[i] == 'F') { // 数值
            // 存入数值栈
        } else if (str[i] == '!') { // 取反
            while (str[i + 1] == ' ') // 省略空格
                i++;
            if (str[i + 1] == '(') // 如果后一个字符是'(', 则取反操作入栈

```

```

        op.push(str[i]);
    } else if (str[i + 1] == 'V' || str[i + 1] == 'F') { // 取反数值
        num.push(str[i + 1] == 'V' ? false : true);
        i++;
    }
} else if (str[i] == '(') { // 左括号直接入栈
    op.push(str[i]);
} else if (str[i] == '|' || str[i] == '&') {
    while (priority(str[i]) <= priority(op.peek()) && op.peek() != '\0' && op.peek() != '(') {
        // 栈顶操作符优先级大于当前操作符，或栈顶是左括号，则弹出、计算栈顶操作符
    }
    op.push(str[i]);
} else if (str[i] == ')') {
    // 右括号，弹出栈顶操作符直到遇到左括号
}
op.pop(); // 弹出 '('
if (op.peek() == '!') {
    // 如果是取反，则计算取反
}
}
}
while (op.peek() != '\0') {
    // 弹出、计算剩余操作符
}
cout << "Expression " << n << ": " << (num.pop() == 1 ? "V" : "F")
    << endl;
}

```

2.3.5 调试分析（遇到的问题 and 解决方法）

在解决布尔表达式计算的问题时，我遇到的最大难点主要在于需要考虑在解析过程中各种符号和操作符出现时所需处理的不同情况。由于布尔表达式的复杂性，短时间内很难全面考虑到所有可能的情形，比如不同操作符的优先级、括号的配对以及空格的处理等。因此，我在编写代码的初期，常常感到困惑和无从下手。

为了解决这个问题，我决定采用一种更为系统的方法。我开始使用测试数据进行反复比对和优化。在每次运行代码时，我都会准备不同的测试用例，包括各种可能的布尔表达式和边界情况。通过对比输出结果与预期结果，我能够快速发现代码中的漏洞和未考虑到的情况。每当我发现一个遗漏的情况，就立即调整代码逻辑，确保能够正确处理所有符号和操作的组合。

这种方法不仅让我逐步修复了代码中的问题，也大大提高了我对布尔表达式解析的理解。随着测试数据的不断增加和代码的逐步优化，我逐渐能够掌握这些复杂符号的处理逻辑，从而使得最终的实现更加健壮。

2.3.6 总结和体会

通过这道题目，我深刻认识到了测试数据的重要性。在以往的编程实践中，我往往只是在代码出现错误后，随意寻找 bug，缺乏系统的测试思维。然而，这道题让我意识到，合理的测试数据可以帮助我快速锁定代码中的问题，提升调试效率。

在编写和测试代码的过程中，我学会了如何设计多样化的测试用例，以覆盖各种可能的情况。这一过程不仅加深了我对布尔运算逻辑的理解，也培养了我解决问题时的系统性思维能力。今后，我将更加注重测试数据的设计与使用，确保在编写代码的同时，能够全面考虑各种可能的情况，从而提高代码的可靠性和可维护性。这种经验将为我后续的学习和项目工作中提供重要的指导，使我在面对复杂问题时更加从容应对。

2.4 队列的应用

2.4.1 问题描述

描述

输入一个 $n \times m$ 的 0 1 矩阵，1 表示该位置有东西，0 表示该位置没有东西。所有四邻域联通的 1 算作一个区域，仅在矩阵边缘联通的不算作区域。求区域数。此算法在细胞计数上会经常用到。对于所有数据， $0 \leq n, m \leq 1000$

2.4.2 基本要求

输入

第 1 行 2 个正整数 n, m ，表示要输入的矩阵行数和列数

第 2— $n+1$ 行为 $n \times m$ 的矩阵，每个元素的值为 0 或 1。

输出

1 行，代表区域数

2.4.3 数据结构设计

图、描述均可

贴出数据结构定义代码（变量、结构体或类）并使用注释辅助说明

```
struct queue { // 广搜队列
    int front, rear, size;
    int* arr; // 队列数组

    queue(int n) { // 构造函数
        ~queue() { // 析构函数
        }

        bool isEmpty() { return front > rear; } // 判断队列是否为空

        void enqueue(int item) { // 入队操作
            if (rear == size - 1) {
                // 扩大队列容量
            }

            // 将元素添加到队列
        }

        int dequeue() { // 出队操作
            if (isEmpty()) {
                cout << "Queue is empty!" << endl;
                return -1;
            }
        }
    }
};
```

```

    }

    // 返回出队的元素

}

};

```

2.4.4 功能说明（函数、类）

- # 推荐使用代码+注释的形式，清晰描述函数功能、输入内容和输出/返回内容
- # 不要直接贴函数实现代码，请选择关键代码/伪代码展示并用注释辅助说明

```

class Solution { // 解决方案类
public:
    Solution() {} // 默认构造函数
private:
    void bfs(queue& q, bool* map, int row, int col, int loc) { // 广度优先搜索方法
        q.enqueue(loc);
        map[loc] = false;
        while (!q.isEmpty()) { // 广搜队列非空时循环
            int current = q.dequeue();
            int current_x = current / col, current_y = current % col; // 计算坐标
            if (current_x > 0 && map[current - col]) { // 上
                q.enqueue(current - col);
                map[current - col] = false;
            }
            // 下、左、右同理
        }
    }
public:
    int solution(bool* map, int row, int col) { // 计算解决方案的主方法
        int res = 0;
        queue q(10); // 广搜队列
        for (int i = 0; i < row * col; i++) {
            if (i / col == 0 || i / col == row - 1 || i % col == 0 ||
                i % col == col - 1) { // 边界不搜索 }
            else { // 如果该位置数值为1，用 bfs 函数搜索 }
        }
        return res; // 返回结果
    }
};

```

2.4.5 调试分析（遇到的问题解决方法）

在解决连通区域计数的问题时，我遇到的主要挑战在于如何有效地搜索和识别连通区域，以及如

何处理那些仅在边界上相连但不算作同一区域的情况。最初，我对如何实现这一逻辑感到困惑，因此我进行了广泛的资料查询和学习，最终了解到这个问题其实是一个经典的算法问题，可以使用深度优先搜索（DFS）或广度优先搜索（BFS）来解决。

为了实现连通区域的搜索，我决定采用广度搜索算法。在实现过程中，我特别注意到边界元素的处理。为了确保只计算在内部相连的区域而不包括边界相连的情况，我采取了一种有效的策略：在进行广度搜索时，直接忽略位于矩阵边缘的元素。这样一来，只有那些位于矩阵内部的元素才会被考虑在内，从而避免了边界相连的干扰。

通过不断测试和调整我的实现，我逐步优化了搜索的逻辑，确保能够准确计算出内部连通区域的数量。这一过程中，我也进一步加深了对深度搜索和广度搜索算法运作机制的理解。

2.4.6 总结和体会

通过这道题目，我不仅学习了深度优先搜索和广度优先搜索算法的基本原理，还初步掌握了它们在实际问题中的应用。这个过程让我认识到，面对复杂问题时，选择合适的算法是解决问题的关键。

此外，在实现过程中，我意识到了对边界条件的处理重要性，尤其是在处理图或矩阵相关的问题时。通过合理设计搜索策略，我能够有效避免常见的错误和遗漏，从而提高了代码的可靠性和准确性。这一经验让我在今后的学习中，将更加注重算法的选择与应用，培养系统化的思维方式，使我在面对更复杂的算法问题时更加自信和从容。总的来说，这次实验让我收获颇丰，为我今后的编程学习奠定了良好的基础。

2.5 队列中的最大值

2.5.1 问题描述

描述

给定一个队列，有下列 3 个基本操作：

- (1) Enqueue(v): v 入队
- (2) Dequeue(): 使队首元素删除，并返回此元素
- (3) GetMax(): 返回队列中的最大元素

请设计一种数据结构和算法，让 GetMax 操作的时间复杂度尽可能地低。

要求运行时间不超过一秒

2.5.2 基本要求

输入

解释

第 1 行 1 个正整数 n，表示队列的容量(队列中最多有 n 个元素)

接着读入多行，每一行执行一个动作。

若输入"dequeue"，表示出队，当队空时，输出一行"Queue is Empty";否则，输出出队的元素；

若输入"enqueue m"，表示将元素 m 入队,当队满时(入队前队列中元素已有 n 个)，输出"Queue is Full"，否则，不输出；

若输入"max",输出队列中最大元素，若队空，输出一行"Queue is Empty"。

若输入"quit",结束输入，输出队列中的所有元素

解释

对于 20%的数据，有 $0 \leq n \leq 100$ ， $|\max(m) - \min(m)| \leq 1e4$ ；

对于 40%的数据，有 $0 \leq n \leq 6000$ ， $|\max(m) - \min(m)| \leq 1e6$ ；（未优化 $O(nm)$ 程序运行时间略微超过一秒）

对于 100%的数据，有 $0 \leq n \leq 10000$ ， $|\max(m) - \min(m)| \leq 1e8$ ；

对于每个测试点， $0x80000000 \leq \min(m) < \max(m) \leq 0x7fffffff$

对于每个测试点，操作的个数约为队列大小的 10 倍左右

事实上，测试数据保证对于后 80%左右的操作，队列内空位不会超过 15%

具体测试数据生成见 p101.py，下载并运行 p101.py 生成随机测试数据

输出

多行，分别是执行每次操作后的结果

2.5.3 数据结构设计

图、描述均可

贴出数据结构定义代码（变量、结构体或类）并使用注释辅助说明

```
typedef int Status;
typedef int ElemType;
ElemType MAX; // 定义最大值
#define FULL 4
#define EMPTY -1
#define OK 1
```

```
struct Stack { // 基础栈结构
    ElemType* base; // 栈底指针
    ElemType* top; // 栈顶指针
    int stacksize; // 栈大小

    Stack(); // 构造函数
    ~Stack(); // 析构函数
```

```
    Status Push(ElemType elem); // 入栈
    Status Pop(ElemType& elem); // 出栈
    Status PeekTop(ElemType& elem); // 获取栈顶元素
    bool StackEmpty(); // 判断栈是否为空
    bool StackFull(); // 判断栈是否已满
};
```

```
class Stack_pro { // 带最大值的栈
private:
    Stack value, max_value; // 栈和最大值栈
public:
    Status Push(ElemType elem); // 入栈
    Status Pop(ElemType& elem); // 出栈
```

```

    Status PeekTop(ElemType& elem) ;      // 获取栈顶元素
    Status GetMaxValue(ElemType& elem) ; // 获取栈中最大值
    bool StackEmpty() ;                  // 判断栈是否为空
    bool StackFull() ;                   // 判断栈是否已满
};

```

```

class Stack_pro_max {
private:
    Stack_pro a,b;
    int num=0; // 队列中元素个数
public:
    Status EnQueue(ElemType elem) ;      // 入队
    Status DeQueue(ElemType& elem) ;      // 出队
    Status GetMaxValue(ElemType& elem) ; // 获取队列中最大值
    bool QueueEmpty() ;                  // 判断队列是否为空
};

```

2.5.4 功能说明（函数、类）

推荐使用代码+注释的形式，清晰描述函数功能、输入内容和输出/返回内容

不要直接贴函数实现代码，请选择关键代码/伪代码展示并用注释辅助说明

```

// 带最大值栈类的弹出操作
Status Stack_pro::Pop(ElemType& elem) {
    ElemType temp;
    if (value.StackEmpty())
        // 栈为空，返回空状态
    value.Pop(elem);
    max_value.PeekTop(temp);
    if (elem == temp)
        // 如果弹出的元素是最大值，则弹出最大值栈
    return OK;
}

```

```

// 带最大值栈类的压入操作
Status Stack_pro::Push(ElemType elem) {
    ElemType temp;
    max_value.PeekTop(temp);

```

```

    if (value.Push(elem) == FULL) {
        return FULL;
    }
    if (max_value.StackEmpty() || elem >= temp)
        // 如果栈为空或新元素大于等于栈顶元素，则压入最大值栈
    return OK;
}

```

```
// 栈队列类获取最大值的方法
Status Stack_pro_max::GetMaxValue(ElemType& elem) {
    elem = 0X80000000; // 最大值初始化为 ElemType 的最小值
    if (num <= 0) {
        return EMPTY;
    }
    ElemType temp=0X80000000; // 临时变量初始化为 ElemType 的最小值
    if (!b.StackEmpty()) { // 如果栈 b 不为空，则获取栈 b 的最大值
        b.GetMaxValue(elem);
    }
    if (!a.StackEmpty()) { // 如果栈 a 不为空，则获取栈 a 的最大值
        a.GetMaxValue(temp);
        elem = (elem > temp) ? elem : temp; // 取最大值
    }
    return OK;
}
```

```
// 栈队列的出队操作
Status Stack_pro_max::DeQueue(ElemType& elem) {
    if (num <= 0) {
        return EMPTY;
    }
    if (b.StackEmpty()) { // 如果栈 b 为空，则将栈 a 中的元素全部弹出并压入栈 b
        ElemType temp;
        while (!a.StackEmpty()) {
            a.Pop(temp);
            b.Push(temp);
        }
    }
    b.Pop(elem);
    num--;
    return OK;
}
```

2.5.5 调试分析（遇到的问题 and 解决方法）

在解决队列中的最大值问题时，主要的挑战在于如何降低获取最大值操作的时间复杂度。直接遍历整个队列来寻找最大值的时间复杂度是 $O(n)$ ，这样的效率显然无法满足需求。因此，我需要寻找一种更高效的解决方案。

通过思考，我决定采用两个栈来模拟一个队列的操作，这样可以有效地维护最大值。其中，栈 A 用于存储进入队列的元素，以保持其入队的顺序，而栈 B 则用于管理出队列的元素。在进行入队操作时，我会将新元素压入栈 A，同时检查栈 A 的当前最大值，并相应地更新栈 B 中的最大值。在出队时，我会从栈 B 中弹出元素，并在必要时更新栈 B 的最大值。通过这种方法，栈 A

和栈 B 分别维护了元素的顺序和最大值，从而保证了获取最大值操作的时间复杂度仅为 $O(1)$ 。在实现过程中，我通过不断测试和调整，确保在各种情况下都能正确地维护和返回最大值。这种双栈的设计不仅提高了代码的运行效率，也让我对数据结构的灵活应用有了更深刻的理解。

2.5.6 总结和体会

通过这个实验，我学习到了用两个栈来模拟队列的方法，并深刻体会到了数据结构设计的巧妙之处。通过合理的数据结构和算法设计，可以显著降低时间复杂度，提升代码的运行效率。这种思维方式不仅适用于当前的问题，也为我在未来的编程学习中提供了新的视角。

此外，我在实践中认识到，选择合适的数据结构不仅能够优化代码性能，还能简化问题的解决过程。今后，我将更加注重对数据结构的深入研究，探索它们在解决实际问题中的广泛应用。这次实验让我收获颇丰，不仅提升了我的编程技能，也强化了我的算法思维，使我在面对复杂问题时能够更加从容应对。

3. 实验总结

本次实验主要围绕栈和队列这两个基本数据结构展开，通过一系列实际问题的解决，我对这两种数据结构的工作原理有了更深入的理解。对于栈，我学习到其遵循先进后出（LIFO）的原则，而队列则遵循先进先出（FIFO）的原则。这种理解让我在解决问题时能够更加灵活地选择合适的数据结构。

在实验过程中，我还发现了栈与队列之间的紧密联系。例如，通过使用两个栈来实现一个队列的功能，不仅可以保持元素的顺序，还能优化获取最大值等操作的时间复杂度。这种创新的思维让我意识到，数据结构的设计并不是单一的，而是可以通过组合和变换来解决更复杂的问题。

此外，我深刻认识到合理的数据结构设计对解决复杂问题的重要性。通过对不同算法和数据结构的应用与优化，我能够有效提升代码的性能，减少运行时间。这种优化思维将为我今后的编程学习和项目开发提供宝贵的指导。

希望在未来的学习中，能够继续深入探索数据结构的广泛应用，特别是在算法设计、数据处理等领域。通过不断实践和学习，我期待能够掌握更多复杂数据结构的使用方法，为解决问题打下坚实的基础。这次实验不仅增强了我的编程能力，也提升了我的逻辑思维和问题解决能力，让我对计算机科学的广阔领域充满了期待。